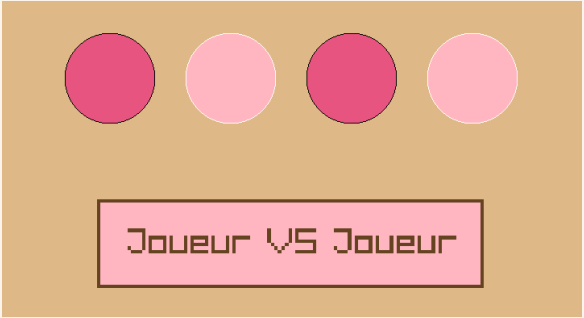
# Projet Othello - L3 Informatique

Julia, Andréa, Lucile, Sarah

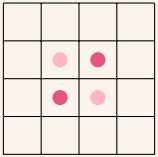
Mai 2026

# Mode joueur contre joueur

- Deux joueurs humains
- Base du projet
- Nécessaire pour l'IA



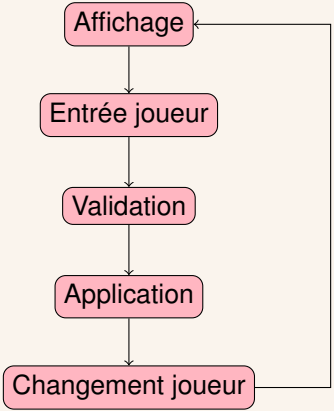
# Plateau 8×8



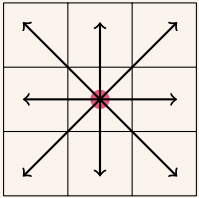
Initialisation

- Matrice 2D
- 3 états (vide / rose foncé / rose clair)

# Boucle du jeu



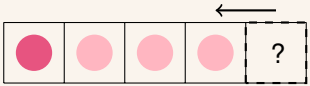
# Validation : 8 directions



Scan directions

- 8 directions testées
- Recherche “sandwich”

# Validation d'un coup (exemple)

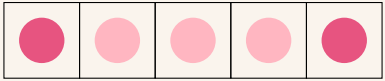



On encadre les pions adverses

- Case voisine adverse obligatoire
- On avance dans la direction
- On doit retrouver notre pion

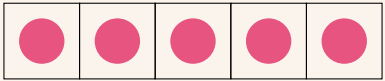
# Principe de capture

**Avant**



Encadrement

**Après**



Changement de couleur

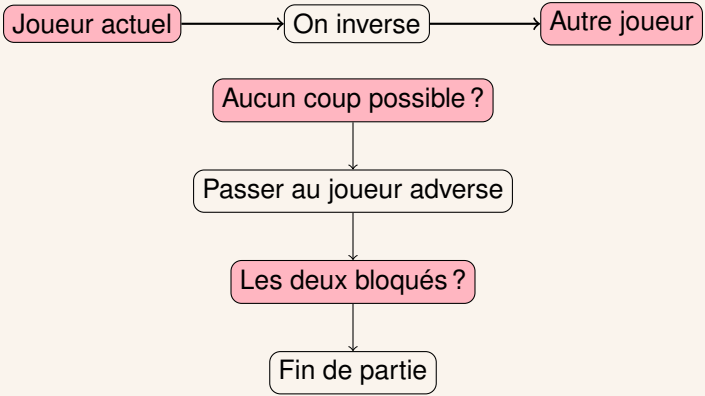
# Mise à jour du score

Compteur rose foncé

Compteur rose clair

- Double boucle sur la matrice 8×8
- Comptage des pions
- Complexité constante

# Changement de joueur / Fin de partie



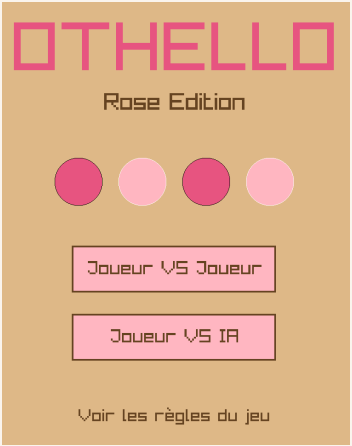

- Comptage final des pions
- Comparaison des scores
- Détermination du gagnant

# Complexité

- Taille fixe du plateau
- Parcours toujours limité
- Complexité en pratique : $O(1)$

# Conclusion Mode joueur contre joueur

Joueur vs Joueur → **Fiabilité du moteur de jeu** → Joueur vs IA

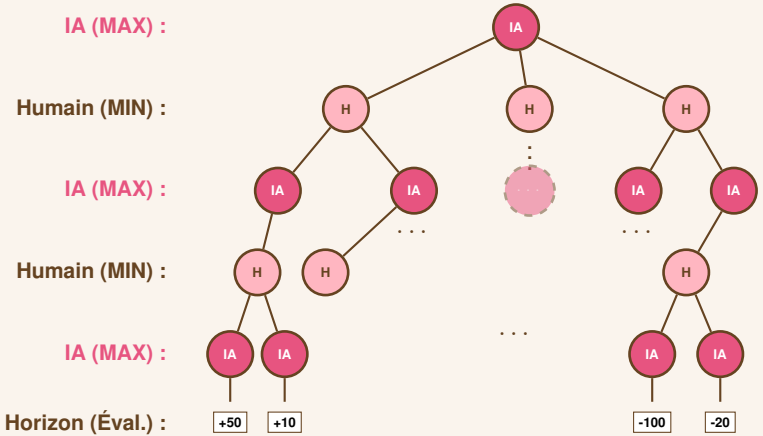


# L'Intelligence Artificielle

Mode : Joueur contre IA

# L'Algorithme Minimax

Simulation de l'arbre de décision



# Fonction d'évaluation



- **Principe :** Attribuer une valeur fixe à chaque case du plateau.
- **Points Forts (+100) :** Les coins sont définitifs et servent d'ancres.
- **Points Faibles (-50) :** Les cases adjacentes aux coins (cases X) risquent de "donner" le coin à l'adversaire.

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 100 | -20 | 10 | 5 | 5 | 10 | -20 | 100 |
| -20 | -50 | -2 | -2 | -2 | -2 | -50 | -20 |
| 10 | -2 | 5 | 1 | 1 | 5 | -2 | 10 |
| 5 | -2 | 1 | 0 | 0 | 1 | -2 | 5 |
| 5 | -2 | 1 | 0 | 0 | 1 | -2 | 5 |
| 10 | -2 | 5 | 1 | 1 | 5 | -2 | 10 |
| -20 | -50 | -2 | -2 | -2 | -2 | -50 | -20 |
| 100 | -20 | 10 | 5 | 5 | 10 | -20 | 100 |

Hiérarchie des cases stratégiques

# Fonction d'évaluation



- **Définition :** C'est le nombre de coups valides qu'un joueur peut jouer à son tour.

**Calcul du score**

$$Score = MesCoupsPossibles - CoupsAdversaire$$

*(Différence des mobilités)*

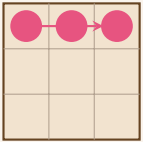
- **L'objectif :** « Étouffer » l'adversaire pour le forcer à jouer sur les cases à malus (adjacentes aux coins).

# Fonction d'évaluation



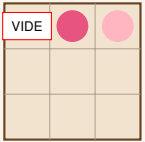
- **Définition :** Un pion est dit « stable » s'il est mathématiquement impossible pour l'adversaire de le retourner d'ici la fin de la partie.
- **La règle :** La stabilité se propage à partir des coins vers le long des bords.

## Cas Stable (Ancré)



*Propagation depuis le coin*

## Cas Instable (Vulnérable)




*Risque de capture (sandwich)*

# Fonction d'évaluation



**En début de partie :**

- **Piège :** Trop de pions créent des « cibles » faciles pour l'adversaire.
- **Risque :** Perdre en mobilité.
- **IA :** Donne un poids **négligeable** au score (Poids 1).

**En fin de partie :**

- **Changement :** L'IA prend en compte le nombre de pions restant dès qu'il reste moins de 10 cases vides.
- **IA :** Le nombre de pions devient un critère **important** (Poids 100).

| Phase de jeu | Coefficient | Stratégie de l'IA |
|---|---|---|
| Début / Milieu | **1** | Privilégier le placement et la mobilité |
| **Fin (10 cases vides)** | **100** | **Priorité au nombre de pions** |

# La Prise de Décision



- **Lister et Trier :** Identifier les coups légaux et privilégier les cases fortes (coins).
- **Simuler :** Tester chaque option sur une copie virtuelle du plateau.
- **Anticiper :** Utiliser le **Minimax** pour prédire le score 5 coups plus tard.
- **Sélectionner :** Retenir le coup qui garantit la meilleure note finale.

# Analyse de la complexité



- **Complexité Minimax :** $O(b^d)$

## Les paramètres

- $b$ : nombre moyen de coups possibles ($\approx 10$).
- $d$ : profondeur de recherche (5).

## Croissance exponentielle

- Pour $d = 5$ : **100 000** positions.
- Pour $d = 6$ : **1 000 000** positions.
- Pour $d = 10$ : **10 milliards**.

**Constat :** Plus on veut une IA intelligente ($d$ élevé), plus le temps de calcul explose.

# Défi : L'explosion combinatoire

Comment augmenter les performances de notre ia sans ralentir le jeu ?

$\rightarrow$ **L'élagage Alpha-Bêta**

# L'Élagage Alpha-Bêta

Optimisation de l'Intelligence Artificielle

# L'algorithme Alpha-Bêta



- **Limite de Minimax :** arbre de jeu très grand $\rightarrow$ coût élevé
- **Idée :** ne pas explorer les branches inutiles
- **Élagage :** arrêt dès qu'une branche n'influence plus la décision
- **Avantage :** même résultat que Minimax mais moins de calculs

**Bornes**

- $\alpha$ : meilleur score pour MAX
- $\beta$ : meilleur score pour MIN

**Condition de coupure :**

$$\alpha \geq \beta$$

# Complexité de l'algorithme Alpha-Bêta

## Complexité

- **Pire cas :** $O(b^d)$
- **Cas moyen :** $O(b^{3d/4})$
- **Meilleur cas :** $O(b^{d/2})$

## Dans notre projet

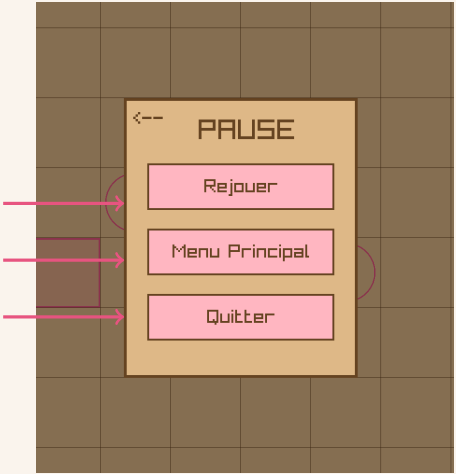
- $b \approx 10$ coups possibles par tour
- Profondeur $d = 5$
- On se situe entre le **cas moyen** et le **meilleur cas**

**Constat :**

Explore 2 fois plus de profondeur que Minimax

# Fonctionnalités du jeu

- **Rejouer :** Relance une nouvelle partie
- **Menu Principal :** Changer de mode de jeu.
- **Quitter :** Propose de sauvegarder



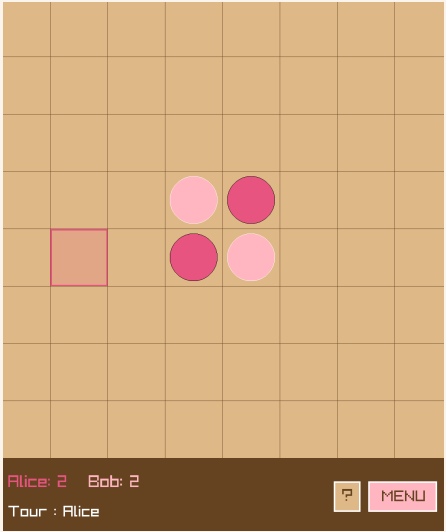
# Système de sauvegarde

1. **Quitter**
2. **Sauvegarder ?**

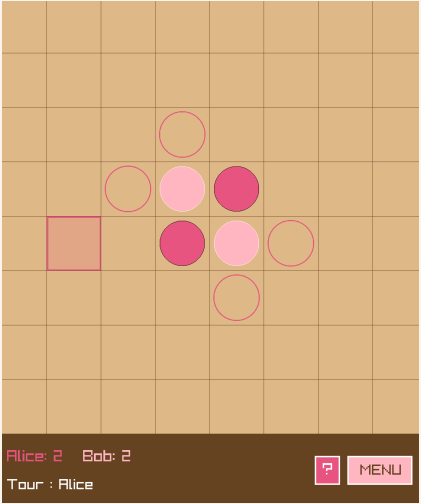
   **Oui :** Partie enregistrée en local

   **Non :** Partie supprimée
3. Option : « **Continuer** »

# Aide visuelle : le bouton « ? »




Bouton « ? »




*Les coups possibles s'affichent*

# Conclusion générale

## Ce qu'on a réalisé

Un jeu Othello complet en C : logique du jeu, IA, interface graphique et fonctionnalités utilisateur.

**Côté technique :**

- Mode Joueur vs Joueur $\rightarrow$ IA
- Minimax + Alpha-Bêta
- A* non adapté

**Côté utilisateur :**

- Interface Raylib
- Sauvegarde locale
- Aide visuelle et menu complet

# Merci pour votre écoute